Software testing classification

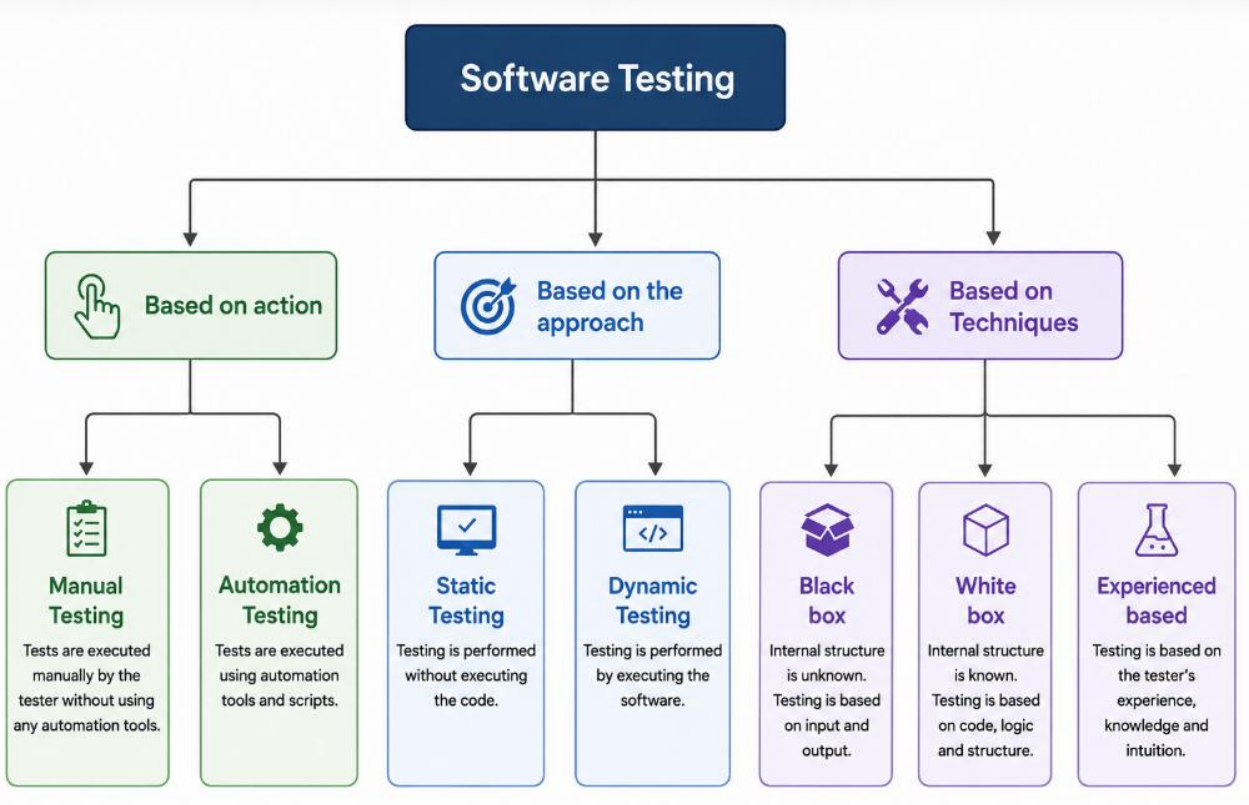


Figure 1 Software Testing Classification

Static testing and Dynamic testing

Static testing is a type of testing where the software is tested **without executing the code or application**.

In static testing, testers, developers, business analysts, or other team members review documents, requirements, designs, test cases, or source code to find defects early.

Use Cases of Static Testing:

1. Requirement Review
2. Design Review
3. Test Case Review
4. Code Review
5. API Specification Review
6. User Story / Acceptance Criteria Review

Dynamic testing is performed when the developed software or product is executed and tested by the tester. In this testing, the tester runs the application, gives input, checks output, and verifies whether the system behaves as expected.

2 types of dynamic testing:

1. **Functional testing:** Testing functionality of a feature
2. **Non-Functional testing**: Testing performance, security, UI/UX etc.

Non functional testing

1. Performance Testing

Performance testing checks how fast, stable, and scalable the system is under different user loads.

Example: For an e-commerce website, test whether 1,000 users can browse products and place orders at the same time without slow response or server crash.

2. Security Testing

Security testing checks whether the system is protected from unauthorized access, data leaks, and common attacks.

Example: For a login API, test whether users can access another user's profile by changing the user ID in the request URL or token.

3. Usability Testing

Clarification: Usability testing checks whether the application is easy, clear, and comfortable for real users to use.

For a registration form, check whether users can easily understand required fields, validation messages, and the final submit process.

4. Cross Browser / Device Testing

Clarification: Cross browser and device testing checks whether the application works correctly across different browsers, screen sizes, and devices.

Test the same checkout page on Chrome, Firefox, Safari, Android mobile, iPhone, tablet, and desktop to ensure layout and functionality remain consistent.

5. Benchmark Testing

Benchmark testing compares the system's performance against a standard, previous version, competitor product, or expected target.

Example: If the current login API response time is 800ms, benchmark testing may check whether the optimized version improves it to the target response time of 300ms.

Test Level

Test level means a specific stage of testing in the software development life cycle where testing is performed with a specific objective. They are connected to other activities in the software development life cycle (SDLC).

There are mainly 4 types of Test level that are:

1. Unit/Component testing
2. Integration Testing
3. System Testing
4. Acceptance Testing

Integration testing has been divided into 2 types:

1. Component Integration Testing
2. System Integration Testing

Component Testing:

1. Component Testing: This level of testing focuses on evaluating individual components or units of a software system to ensure they work correctly in isolation.
2. Component Integration Testing: It involves testing the interactions and interfaces between different components to ensure they integrate seamlessly.

System Testing:

1. System Testing: This level of testing evaluates the entire software system as a whole to verify that it meets its specified requirements and functions correctly.
2. System Integration Testing: It's the testing of interactions and interfaces between different systems or modules to ensure that the overall system functions as expected.

Acceptance Testing:

1. Acceptance Testing: This is the final testing phase where the software is tested by stakeholder to ensure that it meets the business or end-users' requirements and is ready for deployment.

Test Pyramid

The test pyramid is a model that helps with test automation and allocation of testing efforts. It shows that there are different types of tests with varying levels of granularity.

1. The bottom layer consists of small, isolated, and fast tests that check specific functionality. You need many of these tests for good coverage.
2. The top layer includes high-level, end-to-end tests that are slower and cover larger functionality. The higher the layer, the broader and more complex the tests.
3. There are variations in the number and names of layers in different models. For instance, one model has three layers: **unit tests, service tests, and UI tests.**
4. Another model uses **unit tests, integration tests, and end-to-end tests**. Other testing levels can also be employed.

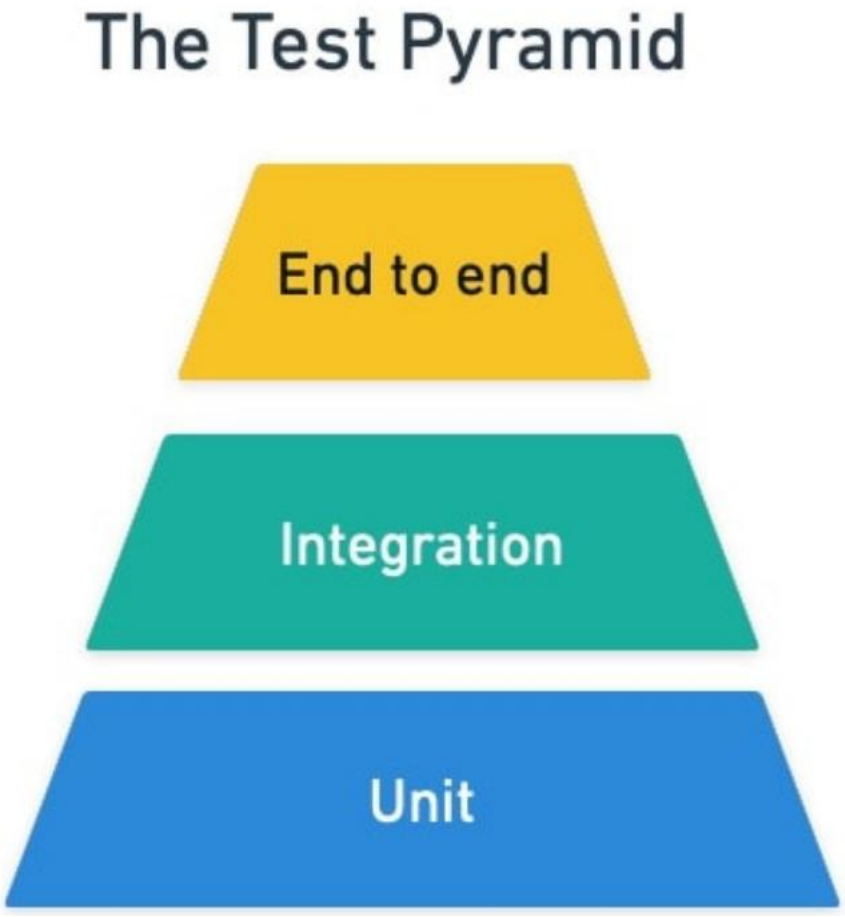


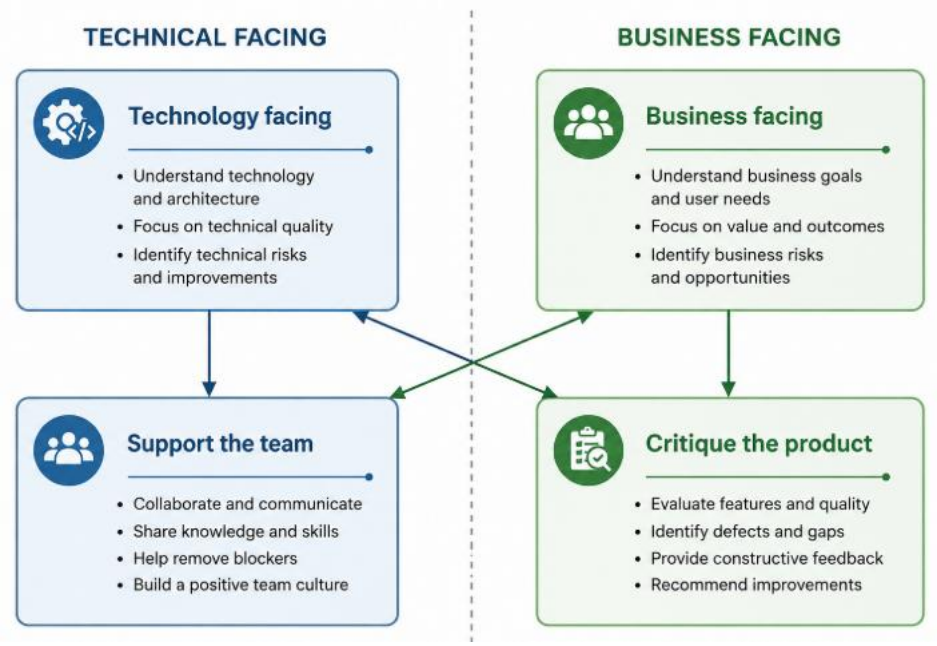
Figure 2: Test Pyramid

Test Quadrant

Testing quadrant **group the test levels** with the appropriate test types, activities, test techniques and work products in the Agile software development.

In this model, tests can be **business facing or technology facing**.

Tests can also **support the team** or **critique the product**



The combination of these two viewpoints determines the four quadrants:

Quadrant Q1 - *Technology Facing, Support the Team*:

- Contains: Component and component integration tests
- Characteristics: **Automated**, included in CI process

Quadrant Q2 - *Business Facing, Support the Team*:

- Contains: Functional tests, user story tests, user experience prototypes, API testing
- Characteristics: Checks acceptance criteria, **manual or automated**

Quadrant Q3 - *Business Facing, Critique the Product*:

- Contains: Exploratory testing, usability testing, user acceptance testing
- Characteristics: User-oriented, **often manual**

Quadrant Q4 - *Technology Facing, Critique the Product*:

- Contains: Smoke tests, sanity test, regression test, system testing, non-functional tests (excluding usability tests)
- Characteristics: **Often automated**